Agente Estadístico Asintótico

Implementación operacional del Capítulo 5:
Teoría Asintótica y Métodos de Remuestreo

Inferencia Estadística para Agentes Inteligentes

Versión 1.0.0 — Mayo 2026

Documento de Arquitectura y Diseño Detallado

Índice

1. Introducción **3**
- 1.1. Propósito del Agente . . . 3
- 1.2. Marco Teórico . . . 3
- 1.3. Arquitectura General . . . 3
- 1.4. Configuración del Agente . . . 5
- 1.5. Resultados de los Módulos . . . 5

2. Módulo 1: Bootstrap (Motor de Intervalos de Confianza) **6**
- 2.1. Fundamento Teórico . . . 6
  - 2.1.1. Consistencia del Bootstrap . . . 6
- 2.2. Intervalos de Confianza Bootstrap . . . 6
  - 2.2.1. IC Percentil (Def 5.3) . . . 6
  - 2.2.2. IC Percentil-t (Def 5.4) . . . 6
- 2.3. Algoritmo Implementado . . . 7
- 2.4. Bootstrap por Bloques para Datos Dependientes . . . 8
- 2.5. Análisis de Complejidad . . . 8

3. Módulo 2: Diagnóstico de Mixing Geométrico **9**
- 3.1. Fundamento Teórico . . . 9
  - 3.1.1. Mixing Geométrico . . . 9
  - 3.1.2. Verificación Empírica . . . 9
- 3.2. Algoritmo Implementado . . . 9
- 3.3. Interpretación de Resultados . . . 10
- 3.4. Caso de Estudio: Cadena de Markov de 2 Estados . . . 10

4. Módulo 3: Descomposición de Incertidumbre Epistémica vs Aleatoria **11**
- 4.1. Fundamento Teórico . . . 11
  - 4.1.1. Definiciones . . . 11
- 4.2. Procedimiento Empírico . . . 11
- 4.3. Modelo de Ajuste . . . 12
- 4.4. Interpretación Práctica . . . 12

5. Módulo 4: Propagación de Incertidumbre via Delta Method Multivariado **13**
- 5.1. Fundamento Teórico . . . 13
  - 5.1.1. Teorema del Método Delta Multivariado . . . 13
  - 5.1.2. Aplicación a $Q^\pi \to V^\pi$ . . . 13
- 5.2. Algoritmo Implementado . . . 13
- 5.3. Soporte para Transformaciones . . . 14
- 5.4. Ejemplo Numérico: MDP de 3 Estados y 2 Acciones . . . 14

6. Módulo 5: Decisión Autónoma de Recolección **15**
- 6.1. Fundamento Teórico . . . 15
  - 6.1.1. Criterio de Decisión . . . 15
  - 6.1.2. Recomendación de Tamaño Muestral . . . 15
- 6.2. Algoritmo Implementado . . . 15
- 6.3. Consideraciones Adicionales . . . 16

7. Módulo 6: Orquestador Integrado y Reporte **17**
7.1. Clase Orquestadora . . . 17
7.2. Comportamiento Adaptativo . . . 17
7.3. Reporte Integrado . . . 17
7.3.1. Ejemplo de Salida . . . 18

8. Supuestos Críticos y Limitaciones **19**
8.1. Supuestos No Verificables . . . 19
8.2. Limitaciones Prácticas . . . 19
8.3. Recomendaciones de Uso . . . 19

9. Mapeo Completo entre Teoría e Implementación **20**
9.1. Correspondencia Detallada . . . 20
9.2. Diagrama de Dependencias Teóricas . . . 20

10. Casos de Uso y Ejecución **21**
10.1. Caso SSCH (Sistema de Soporte Clínico de Hígado) . . . 21
10.2. Validación Cruzada con Valores del Capítulo . . . 21
10.3. Caso con Dependencia Temporal . . . 21
10.4. Caso con Delta Method . . . 21

11. Conclusiones **21**
11.1. Resumen de Capacidades . . . 21
11.2. Fortalezas del Diseño . . . 22
11.3. Líneas de Extensión Futura . . . 22

1. Introducción

1.1. Propósito del Agente

El **Agente Estadístico Asintótico** es un sistema computacional diseñado para evaluar políticas de decisión en entornos de aprendizaje por refuerzo (RL) desde una perspectiva de inferencia estadística rigurosa. Su función principal es cuantificar la incertidumbre en la estimación del valor esperado de una política V^π y tomar decisiones autónomas sobre si la precisión alcanzada es suficiente para actuar o si se requiere recolectar más datos.

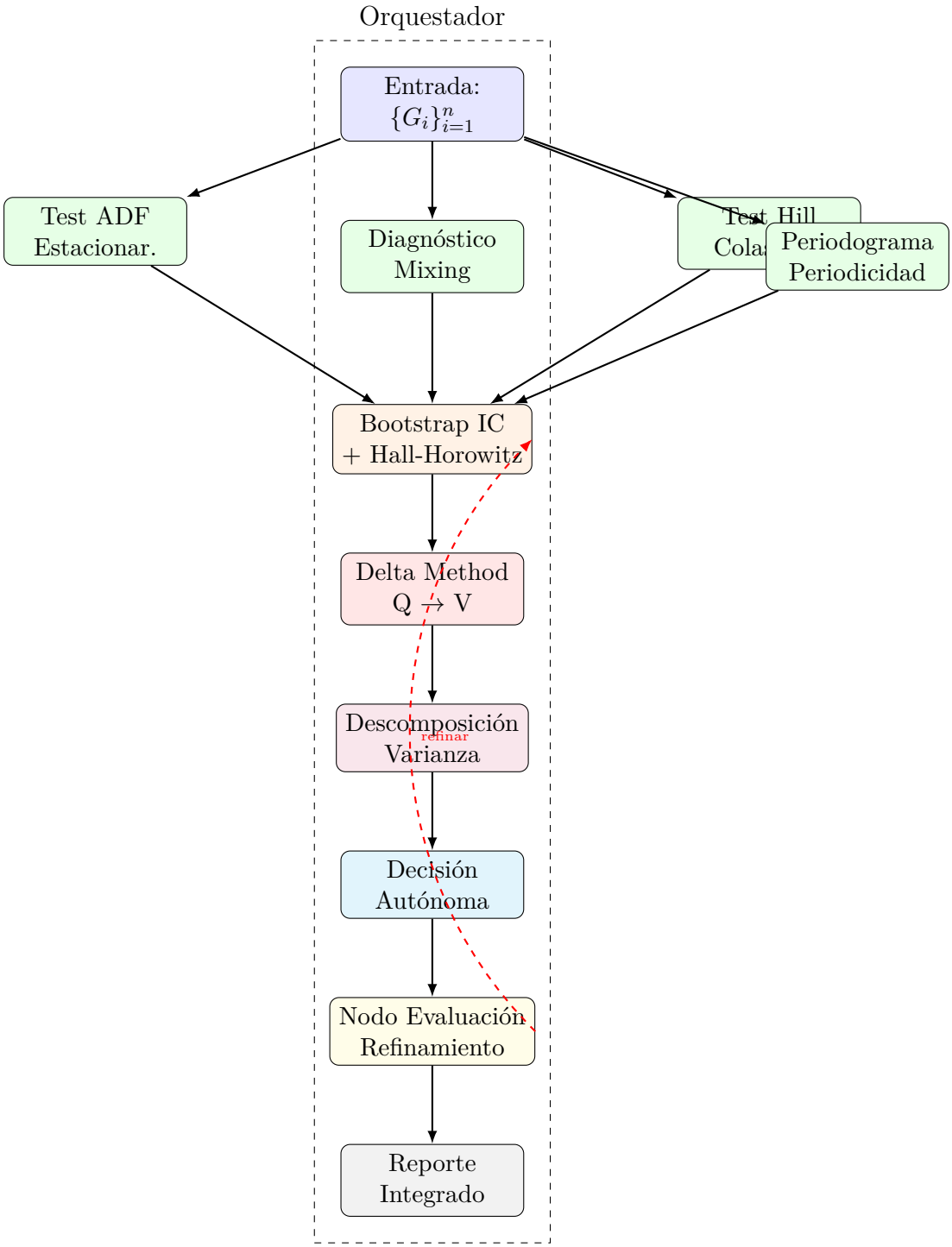
1.2. Marco Teórico

El agente implementa operacionalmente los conceptos del Capítulo 5 de *Inferencia Estadística para Agentes Inteligentes*, que cubre:

- **Teorema Central del Límite (CLT)** clásico y para procesos mixing (Thm 5.1, Thm 5.8).
- **Método Delta** univariado y multivariado (Thm 5.4, Thm 5.5).
- **Bootstrap no paramétrico** y su consistencia (Thm 5.7).
- **Intervalos de Confianza** Percentil (Def 5.3) y Percentil-t (Def 5.4).
- **Diagnóstico de mixing geométrico** (Prop 5.2).
- **Descomposición epistémica vs aleatoria** (Lab F.5.2).
- **Criterio de ancho para decisión autónoma** (Lab F.5.1).

1.3. Arquitectura General

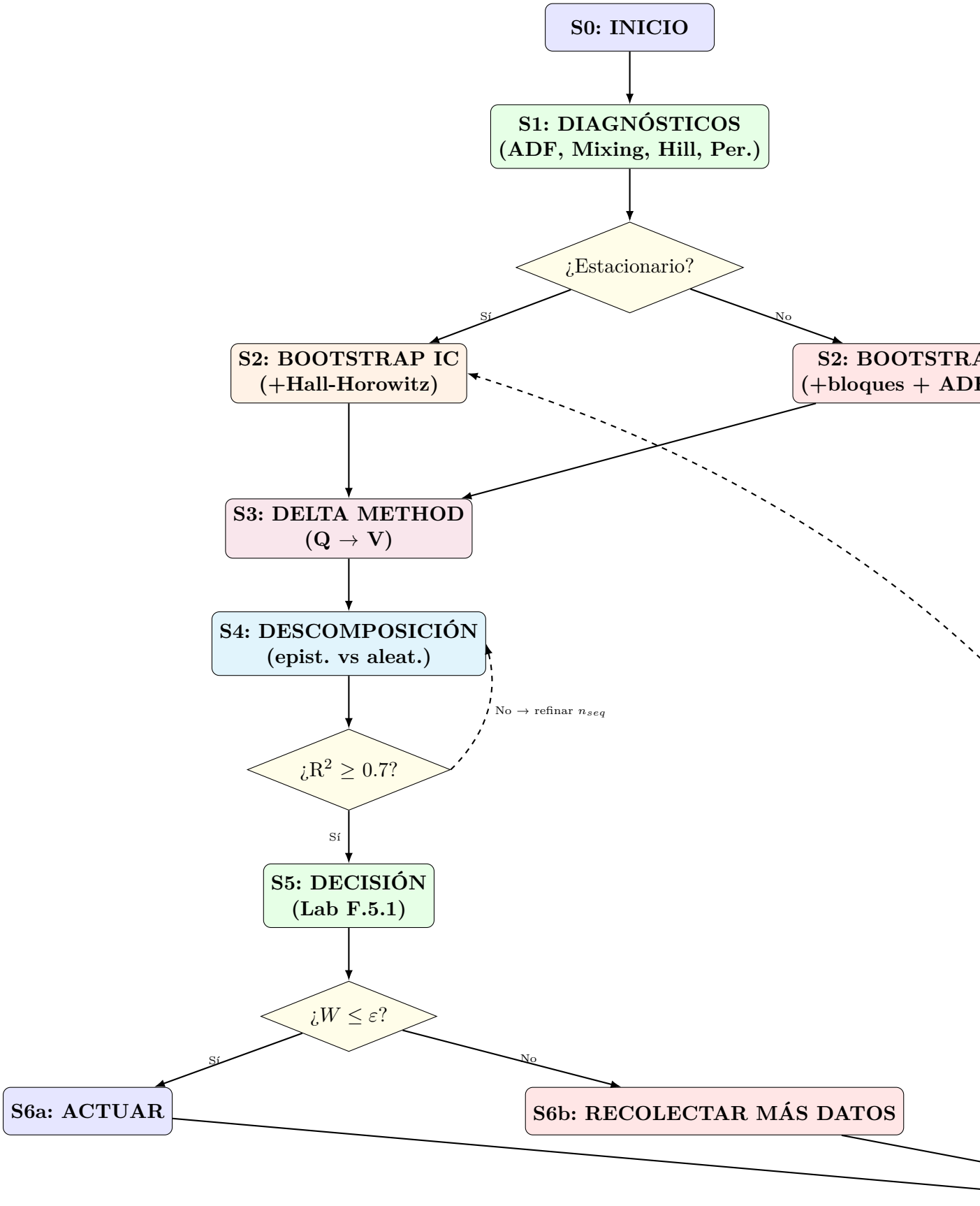
El agente sigue una arquitectura de **grafo orientado acíclico (DAG) con nodo de realimentación** donde los módulos independientes se ejecutan en paralelo y un bucle de refinamiento evalúa la calidad del resultado y decide si repetir pasos anteriores con parámetros ajustados.



El flujo comienza con 4 diagnósticos en paralelo (ADF, Mixing, Hill, Periodograma). Luego el bootstrap integra Hall-Horowitz para el tamaño de bloque. Tras la decisión, el Nodo de Evaluación puede retroalimentar al bootstrap para refinar parámetros (B, ℓ, n).

1.4. StatGraph del Agente

El siguiente diagrama modela el agente como una máquina de estados finitos donde cada nodo es un estado computacional y las aristas son transiciones condicionales:


S0: INICIO
S1: DIAGNÓSTICOS
(ADF, Mixing, Hill, Per.)
¿Estacionario?
Sí
No
S2: BOOTSTRAP IC
(+Hall-Horowitz)
S2: BOOTSTRA
(+bloques + ADI
S3: DELTA METHOD
(Q → V)
S4: DESCOMPOSICIÓN
(epist. vs aleat.)
No → refinar n_{seq}
¿$R^2 \geq 0.7$?
Sí
S5: DECISIÓN
(Lab F.5.1)
¿$W \leq \varepsilon$?
Sí
No
S6a: ACTUAR
S6b: RECOLECTAR MÁS DATOS

El agente comienza en S0, ejecuta 4 diagnósticos en paralelo (S1), decide la estrategia bootstrap según estacionariedad (D1), propaga incertidumbre vía Delta Method si hay Q (S3, opcional), descompone varianza (S4), verifica calidad del ajuste (D2), decide si actuar o recolectar (S5–D3), y finalmente evalúa si necesita refinar parámetros (S7–D4) antes de emitir el reporte terminal (S8).

El agente utiliza `dataclasses` de Python como contratos formales de entrada y salida. Cada dataclass encapsula un conjunto de parámetros con tipos explícitos, validación y documentación asociada.

1.5. Configuración del Agente

Listing 1: ConfiguracionAgente – Parámetros globales del sistema

```python
@dataclass
class ConfiguracionAgente:
    alpha: float = 0.05                    # Nivel de significancia
    B: int = 1000                          # Réplicas bootstrap
    epsilon_tolerancia: float = 0.5        # Umbral de ancho IC
    tipo_ic: Literal["percentil", "percentil-t"] = "percentil-t"
    max_lag_mixing: int = 20               # Máximo lag para
       autocorrelación
    umbral_mixing: float = 0.95            # Umbral de mixing
    block_size: Union[int, None] = None # Bootstrap por bloques
    semilla: Union[int, None] = 42         # Reproducibilidad
```

La tabla 1 resume los parámetros de configuración, sus valores por defecto y las referencias teóricas que los justifican.

Cuadro 1: Parámetros de configuración del agente.

Parámetro	Valor por defecto	Referencia	Descripción
α	0.05	Def 5.3, Def 5.4	Nivel de significancia (IC al 95 %)
B	1000	Thm 5.7	Réplicas bootstrap ($B \geq 1000$)
ε	0.5	Lab F.5.1	Tolerancia máxima de ancho de IC
`tipo_ic`	percentil-t	Def 5.4	Refinamiento de segundo orden
$K_{\text{máx}}$	20	Prop 5.2	Lags máximos para autocorrelación
ρ_{umbral}	0.95	Prop 5.2	Umbral de mixing geométrico
ℓ	None	Tab 5.1	Tamaño de bloque (None = iid)

1.6. Resultados de los Módulos

Cada módulo produce un objeto dataclass que encapsula su salida. La tabla 2 enumera las propiedades principales de cada resultado.

Cuadro 2: Estructuras de datos de salida de cada módulo.

Dataclass	Propiedades clave	Módulo origen
`ResultadoBootstrap`	$\hat{V}$, L, U, `ancho`, `se_bootstrap`, $\{\hat{V}^{*(b)}\}$	Bootstrap
`ResultadoMixing`	$\hat{\rho}$, `mixing_ok`, σ^2_{LR}, $\{\hat{\gamma}_k\}$	Mixing
`ResultadoDescomposicion`	c_{ep}, σ^2_{al}, $f_{\text{ep}}(n)$, n_{rec}	Descomposición
`ResultadoDelta`	$\tilde{V}$, σ^2_V, IC_{95}, $\mathbf{J}$, contribución cov	Delta Method
`ResultadoDecision`	`.ªctuar"` / `recolectar_mas"`, $n_{\text{recomendado}}$	Decisión
`ReporteAgente`	Reporte integrado con garantías teóricas	Orquestador

2. Módulo 1: Bootstrap (Motor de Intervalos de Confianza)

2.1. Fundamento Teórico

El bootstrap no paramétrico, introducido por Efron (1979), aproxima la distribución muestral de un estadístico mediante remuestreo con reemplazo de los datos originales. El Teorema 5.7 (`thm:ch5-bootstrap-media`) garantiza la consistencia del bootstrap para la media muestral bajo condiciones de momentos finitos.

2.1.1. Consistencia del Bootstrap

Teorema 2.1 (Consistencia del Bootstrap para la media, Thm 5.7)**.** *Sea $\{G_i\}_{i=1}^n$ una muestra iid con $\mathbb{E}[G^2] < \infty$. Sean $\hat{V}_n = \bar{G}_n$ y $\hat{V}_n^* = \bar{G}_n^*$ la media bootstrap. Entonces:*

$$\sup_{x\in\mathbb{R}} \left| P^*\left(\sqrt{n}(\hat{V}_n^* - \hat{V}_n) \le x\right) - P\left(\sqrt{n}(\hat{V}_n - V) \le x\right)\right| \xrightarrow{p} 0$$

donde P^* denota probabilidad condicional en la muestra.

2.2. Intervalos de Confianza Bootstrap

2.2.1. IC Percentil (Def 5.3)

El IC percentil utiliza directamente los cuantiles de la distribución bootstrap:

Definición 2.2 (IC Percentil Bootstrap)**.**

$$IC_{1-\alpha} = \left[\hat{V}^{*(B)}_{\lfloor B\cdot\alpha/2\rfloor},\ \hat{V}^{*(B)}_{\lfloor B\cdot(1-\alpha/2)\rfloor}\right]$$

donde $\hat{V}^{*(B)}_{(k)}$ es el k-ésimo estadístico de orden de las B réplicas bootstrap.

Error de cobertura: $O(n^{-1/2})$.

2.2.2. IC Percentil-t (Def 5.4)

El IC percentil-t utiliza un pivote studentizado que proporciona un refinamiento de segundo orden:

Definición 2.3 (IC Percentil-t Bootstrap)**.** *Sea el pivote bootstrap studentizado:*

$$t^{*(b)} = \frac{\hat{V}^{*(b)} - \hat{V}}{SE^{*(b)}}$$

donde $SE^{*(b)}$ es el error estándar bootstrap de la b-ésima réplica, estimado mediante un bootstrap interno (doble bootstrap).

El IC se construye como:

$$IC_{1-\alpha} = \left[\hat{V} - t^*_{(1-\alpha/2)} \cdot SE,\ \hat{V} - t^*_{(\alpha/2)} \cdot SE\right]$$

donde $t^*_{(q)}$ es el q-ésimo cuantil de $\{t^{*(b)}\}_{b=1}^{B}$.

Error de cobertura: $O(n^{-1})$ (refinamiento de segundo orden).

2.3. Algoritmo Implementado

El algoritmo 2 describe la implementación completa del módulo de bootstrap.

Listing 2: Algoritmo de bootstrap completo (código simplificado)

```
def bootstrap_ci(retornos, B=1000, alpha=0.05,
                 tipo="percentil-t", block_size=None, semilla=None):

    n = len(retornos)
    V_hat = np.mean(retornos)
    replicas = np.empty(B)

    # 1. Generar réplicas bootstrap
    for b in range(B):
        if block_size is None:
            # Bootstrap clásico iid
            idx = rng.integers(0, n, size=n)
        else:
            # Bootstrap por bloques circulares
            n_blocks = ceil(n / block_size)
            start_indices = rng.integers(0, n, size=n_blocks)
            # Construir muestra por bloques
            ...
        replicas[b] = np.mean(retornos[idx])

    se_boot = np.std(replicas, ddof=1)

    # 2. Construir IC según tipo
    if tipo == "percentil":
        L = percentile(replicas, 100 * alpha / 2)
        U = percentile(replicas, 100 * (1 - alpha / 2))
    else:  # percentil-t
        t_star = np.empty(B)
        n_inner = max(50, n // 2)
        B_inner = min(200, B // 5)

        for b in range(B):
```

```
            # Doble bootstrap: error estándar interno
            idx_inner = rng.integers(0, n, size=n_inner)
            muestra_inner = retornos[idx_inner]
            # B_inner réplicas para estimar SE
            ...
            se_inner = np.std(replicates_inner, ddof=1)
            t_star[b] = (replicas[b] - V_hat) / se_inner

        q_low = percentile(t_star, 100 * alpha / 2)
        q_high = percentile(t_star, 100 * (1 - alpha / 2))
        L = V_hat - q_high * se_boot
        U = V_hat - q_low * se_boot

    return ResultadoBootstrap(V_hat, L, U, ...)
```

2.4. Bootstrap por Bloques para Datos Dependientes

Cuando el diagnóstico de mixing indica dependencia temporal, el módulo soporta bootstrap por bloques circulares (Tab 5.1). Bajo la condición $\ell_n \to \infty$ y $\ell_n/n \to 0$, este método preserva la estructura de dependencia dentro de cada bloque.

2.5. Análisis de Complejidad

Cuadro 3: Complejidad computacional de los métodos bootstrap.

Método	Réplicas	Complejidad
Percentil	B	$O(B \cdot n)$
Percentil-t	$B \cdot B_{\text{inner}}$	$O(B \cdot n^2)$
Por bloques	B	$O(B \cdot n \cdot \ell)$

3. Módulo 2: Diagnóstico de Mixing Geométrico

3.1. Fundamento Teórico

El mixing geométrico es una condición de dependencia débil que garantiza la validez del CLT para procesos estacionarios. La Proposición 5.2 (`prop:ch5-verificacion`) proporciona un procedimiento empírico para verificar esta condición.

3.1.1. Mixing Geométrico

Definición 3.1 (Mixing Geométrico, Def 5.2)**.** *Un proceso estacionario $\{X_t\}$ es geométricamente mixing si existen constantes $C > 0$ y $\rho \in (0, 1)$ tales que:*

$$\alpha(k) \leq C\rho^k \quad \forall k \geq 1$$

donde $\alpha(k)$ es el coeficiente de mixing fuerte.

3.1.2. Verificación Empírica

Proposición 3.2 (Verificación de Mixing por Autocorrelación, Prop 5.2)**.** *Bajo mixing geométrico, las autocovarianzas decaen exponencialmente:*

$$|\gamma_k| \leq C\rho^k \quad \Rightarrow \quad \log|\gamma_k| \approx \beta_0 + \beta_1 k$$

con $\rho = e^{\beta_1}$. Si $\hat{\rho} < 0{,}95$, el mixing se considera verificado.

3.2. Algoritmo Implementado

El módulo ejecuta los siguientes pasos:

1. **Cálculo de autocovarianzas muestrales**:

$$\hat{\gamma}_k = \frac{1}{n}\sum_{t=1}^{n-k}(X_t - \bar{X})(X_{t+k} - \bar{X}), \quad k = 0, 1, \ldots, K$$

2. **Regresión log-lineal**:

$$\log|\hat{\gamma}_k| = \beta_0 + \beta_1 k + \varepsilon_k$$

   Ajuste por mínimos cuadrados usando solo lags con $\hat{\gamma}_k > 10^{-12}$.

3. **Estimación del coeficiente de decaimiento**:

$$\hat{\rho} = \exp(\hat{\beta}_1)$$

4. **Criterio de decisión**:

$$\text{mixing_ok} = \begin{cases} \text{True} & \text{si } \hat{\rho} < 0{,}95 \\ \text{False} & \text{en caso contrario} \end{cases}$$

5. **Varianza de largo plazo** (estimador de Newey-West truncado):

$$\hat{\sigma}^2_{LR} = \hat{\gamma}_0 + 2\sum_{k=1}^{K}\hat{\gamma}_k$$

3.3. Interpretación de Resultados

- **Si mixing_ok = True**: El CLT para procesos mixing (Thm 5.8) es aplicable. La varianza asintótica del estimador es σ^2_{LR}/n.
- **Si mixing_ok = False**: Se activa automáticamente el bootstrap por bloques como precaución, con tamaño de bloque heurístico $\ell \approx n^{1/3}$.

3.4. Caso de Estudio: Cadena de Markov de 2 Estados

La función `simular_cadena_markov_2estados()` genera datos del ejemplo `ex:ch5-cadena`:

- Matriz de transición: $P = \begin{pmatrix} 0{,}8 & 0{,}2 \\ 0{,}3 & 0{,}7 \end{pmatrix}$
- Recompensas: $r = [0, 10]$
- Distribución estacionaria: $\mu = [0{,}6, 0{,}4]$
- Autocorrelación teórica: ρ^k con $\rho = 0{,}5$

4. Módulo 3: Descomposición de Incertidumbre Epistémica vs Aleatoria

4.1. Fundamento Teórico

El Laboratorio F.5.2 introduce un procedimiento empírico para separar la varianza total del estimador en dos componentes con interpretaciones radicalmente diferentes.

4.1.1. Definiciones

Definición 4.1 (Incertidumbre Epistémica)**.** *Componente de la varianza que se reduce al aumentar el tamaño muestral:*

$$Var_{ep}(\hat{V}_n) = \frac{c_{ep}}{n}$$

Es ***reducible*** *mediante la recolección de más datos.*

Definición 4.2 (Incertidumbre Aleatoria)**.** *Componente irreducible de la varianza, inherente a la estocasticidad del entorno:*

$$Var_{al}(\hat{V}_n) = \sigma^2_{al}$$

No desaparece aunque $n \to \infty$*.*

4.2. Procedimiento Empírico

El algoritmo 3 describe el procedimiento:

Listing 3: Algoritmo de descomposición de incertidumbre

```
def descomponer_incertidumbre(V_hat, retornos, n_seq=None, B=500,
    ...):
    if n_seq is None:
        n_seq = [50, 100, 200, 500, 1000, 2000, 5000]

    varianzas = {}
    for n_k in n_seq:
        if n_k > N: continue
        # Submuestreo aleatorio
        idx = rng.choice(N, size=n_k, replace=False)
        sub = retornos[idx]
        # Bootstrap para estimar Var(V_hat_{n_k})
        r = bootstrap_ci(sub, B=B, alpha=alpha, tipo="percentil")
        varianzas[n_k] = r.se_bootstrap ** 2

    # Ajuste: Var_n = a/n + b/n^2
    def modelo(n, a, b):
        return a / n + b / (n ** 2)

    (a_fit, b_fit), _ = curve_fit(modelo, ns_arr, vars_arr)
    sigma2_ep = a_fit  # Componente epistémica

    return ResultadoDescomposicion(sigma2_ep, ...)
```

4.3. Modelo de Ajuste

El modelo ajustado es:

$$\widehat{\mathrm{Var}}(\hat{V}_n) = \frac{c_1}{n} + \frac{c_2}{n^2}$$

donde:

- c_1/n: Componente epistémica dominante (decrece como $1/n$).
- c_2/n^2: Corrección de segundo orden (captura efectos de cola).

La fracción epistémica al tamaño n actual es:

$$f_{\mathrm{ep}}(n) = \frac{c_1/n}{\widehat{\mathrm{Var}}(\hat{V}_n)}$$

La recomendación de n para reducir la componente epistémica al 10 % de la varianza total es:

$$n_{\mathrm{rec}} = \left\lceil \frac{c_1}{0{,}10 \cdot \widehat{\mathrm{Var}}(\hat{V}_{n_{\mathrm{actual}}})} \right\rceil$$

4.4. Interpretación Práctica

- Si $f_{\mathrm{ep}}(n) \approx 1$: Casi toda la incertidumbre es epistémica. Recolectar más datos reducirá drásticamente la varianza.
- Si $f_{\mathrm{ep}}(n) \approx 0$: La incertidumbre es mayormente aleatoria. Incrementar n tendrá poco efecto.

5. Módulo 4: Propagación de Incertidumbre via Delta Method Multivariado

5.1. Fundamento Teórico

El Método Delta multivariado (Thm 5.5) permite propagar la incertidumbre desde estimaciones de las funciones Q^π (valores estado-acción) hasta el valor del estado V^π.

5.1.1. Teorema del Método Delta Multivariado

Teorema 5.1 (Método Delta Multivariado, Thm 5.5)**.** *Sea $\hat{\boldsymbol{\theta}}_n$ un estimador de $\boldsymbol{\theta} \in \mathbb{R}^d$ que satisface:*

$$\sqrt{n}(\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}) \xrightarrow{d} N(\mathbf{0}, \boldsymbol{\Sigma})$$

Sea $g : \mathbb{R}^d \to \mathbb{R}$ diferenciable en $\boldsymbol{\theta}$ con gradiente $\nabla g(\boldsymbol{\theta}) \neq \mathbf{0}$. Entonces:

$$\sqrt{n}\big(g(\hat{\boldsymbol{\theta}}_n) - g(\boldsymbol{\theta})\big) \xrightarrow{d} N\big(0,\ \nabla g(\boldsymbol{\theta})^\top \boldsymbol{\Sigma}\, \nabla g(\boldsymbol{\theta})\big)$$

5.1.2. Aplicación a $Q^\pi \to V^\pi$

En el contexto del agente (`prop:ch5-delta-q`), la función de agregación es lineal:

$$V^\pi(s) = g(\mathbf{Q}^\pi(s, \cdot)) = \sum_{a \in \mathcal{A}} \pi(a|s) \cdot Q^\pi(s, a)$$

Por lo tanto, el Jacobiano es simplemente el vector de probabilidades de la política:

$$\mathbf{J} = \nabla g = [\pi(a_1|s),\ \pi(a_2|s),\ \ldots,\ \pi(a_{|\mathcal{A}|}|s)]$$

Y la varianza asintótica de $\hat{V}^\pi$ es:

$$\sigma_V^2 = \mathbf{J}^\top \boldsymbol{\Sigma}_Q \mathbf{J} = \sum_{i,j} \pi_i \pi_j \Sigma_{Q,ij}$$

5.2. Algoritmo Implementado

Listing 4: Algoritmo de propagación Delta Method

```python
def propagar_desde_Q(Q_estimates, policy, Sigma_Q, n, alpha=0.05,
                     transformacion="identidad"):

    # Jacobiano = vector de política (agregación lineal)
    J = policy  # shape (|A|,)

    # Varianza asintótica: sigma2_V = J @ Sigma_Q @ J^T
    sigma2_V = float(J @ Sigma_Q @ J)

    # Contribución de covarianzas
    var_puras = sum(policy[i]**2 * Sigma_Q[i,i] for i in range(len(
       policy)))
    contribucion_cov = sigma2_V - var_puras

```

```
    # Estimación puntual
    V_hat = np.dot(policy, Q_estimates)

    # Aplicar transformación (si aplica)
    if transformacion == "exp":
        V_transformado = exp(V_hat)
        derivada = exp(V_hat)
        sigma2_final = derivada**2 * sigma2_V / n
    elif transformacion == "identidad":
        V_transformado = V_hat
        sigma2_final = sigma2_V / n

    # IC asintótico normal
    z = stats.norm.ppf(1 - alpha/2)
    L = V_transformado - z * sqrt(sigma2_final)
    U = V_transformado + z * sqrt(sigma2_final)

    return ResultadoDelta(V_transformado, sigma2_final, (L, U), ...)
```

5.3. Soporte para Transformaciones

El módulo soporta tres tipos de transformación final sobre V:

- `ïdentidad"`: Sin transformación, $g(V) = V$.
- `.ᵉˣp"`: Transformación exponencial, $g(V) = e^V$. Utiliza el Delta Method univariado (Thm 5.4): $\text{Var}(e^{\hat{V}}) \approx e^{2V} \cdot \text{Var}(\hat{V})$.
- `callable`: Función personalizada proporcionada por el usuario.

5.4. Ejemplo Numérico: MDP de 3 Estados y 2 Acciones

El ejemplo `ex:ch5-qdelta` utiliza:

$$\boldsymbol{\Sigma}_Q = \frac{1}{n}\begin{pmatrix} 4{,}0 & 1{,}2 \\ 1{,}2 & 9{,}0 \end{pmatrix}, \quad \mathbf{Q} = \begin{pmatrix} 5{,}0 \\ 4{,}5 \end{pmatrix}, \quad \pi = \begin{pmatrix} 0{,}6 \\ 0{,}4 \end{pmatrix}$$

Resultados teóricos:

$$\begin{aligned} V^\pi &= 0{,}6 \cdot 5{,}0 + 0{,}4 \cdot 4{,}5 = 4{,}8 \\ \sigma_V^2 &= (0{,}6)^2 \cdot 4{,}0 + (0{,}4)^2 \cdot 9{,}0 + 2 \cdot 0{,}6 \cdot 0{,}4 \cdot 1{,}2 \\ &= 1{,}44 + 1{,}44 + 0{,}576 = 3{,}456 \\ \text{SE}(\hat{V}) &= \sqrt{3{,}456/n} \end{aligned}$$

6. Módulo 5: Decisión Autónoma de Recolección

6.1. Fundamento Teórico

El Laboratorio F.5.1 establece un criterio objetivo para que el agente decida autónomamente si la precisión es suficiente.

6.1.1. Criterio de Decisión

Definición 6.1 (Criterio de Decisión, Lab F.5.1)**.** *Dado un intervalo de confianza* $[L, U]$ *para* V^π *con nivel* $1-\alpha$ *y una tolerancia* $\varepsilon > 0$*:*

$$\text{Decisión} = \begin{cases} \texttt{actuar} & \text{si } U - L \leq \varepsilon \\ \texttt{recolectar_más} & \text{si } U - L > \varepsilon \end{cases}$$

6.1.2. Recomendación de Tamaño Muestral

Cuando se decide recolectar más datos, el módulo estima el número adicional de trayectorias necesario utilizando la escala $O(1/\sqrt{n})$ del CLT:

Proposición 6.2. *Bajo el CLT, el ancho del IC escala como:*

$$W_n \approx 2 \cdot z_{1-\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}$$

Por lo tanto, el tamaño necesario para alcanzar un ancho ε *es:*

$$n_{nuevo} \approx n_{actual} \cdot \left(\frac{W_{actual}}{\varepsilon} \right)^2$$

6.2. Algoritmo Implementado

Listing 5: Algoritmo de decisión autónoma

```
def decidir_recoleccion(IC_L, IC_U, epsilon_tolerancia, n_actual,
    ...):
    W = IC_U - IC_L

    if W <= epsilon_tolerancia:
        return ResultadoDecision(
            decision="actuar",
            ancho_ic=W,
            n_recomendado=None,
            justificacion=f"Ancho␣IC␣({W:.4f})␣<=␣tolerancia␣({
                epsilon_tolerancia:.4f})"
        )
    else:
        n_rec = ceil(n_actual * (W / epsilon_tolerancia) ** 2)
        n_rec = max(n_rec, n_actual + 1)
        return ResultadoDecision(
            decision="recolectar_mas",
            ancho_ic=W,
```

```
            n_recomendado=n_rec,
            justificacion=f"Se␣recomiendan␣~{n_rec}␣trayectorias␣
               adicionales"
        )
```

6.3. Consideraciones Adicionales

- La recomendación de n es una cota inferior. En la práctica, puede necesitarse un 10 %–20 % adicional para compensar la variabilidad del estimador de varianza.
- Si la descomposición de varianza está disponible, la componente epistémica se utiliza para refinar la recomendación.
- El parámetro ε es externo al agente y debe ser proporcionado por el usuario según los requisitos de precisión de la aplicación.

7. Módulo 6: Orquestador Integrado y Reporte

7.1. Clase Orquestadora

La clase `AgenteEstadisticoAsintotico` orquesta el pipeline completo:

Listing 6: Orquestador principal – método evaluar_politica

```
class AgenteEstadisticoAsintotico:
    def evaluar_politica(self, retornos, Q_estimates=None,
                         policy=None, Sigma_Q=None):

        # 1. Estimación puntual
        V_hat = np.mean(retornos)

        # 2. Diagnóstico de mixing
        res_mixing = diagnosticar_mixing(retornos, ...)

        # 3. Bootstrap (con bloque si mixing falla)
        if not res_mixing.mixing_ok:
            block_size = ceil(n ** (1/3))  # heurística
        res_boot = bootstrap_ci(retornos, block_size=block_size, ...)

        # 4. Delta Method (si se proveen Q, policy, Sigma_Q)
        if all_provided:
            res_delta = propagar_desde_Q(Q_estimates, policy, Sigma_Q
                , ...)

        # 5. Descomposición de varianza
        res_desc = descomponer_incertidumbre(V_hat, retornos, ...)

        # 6. Decisión autónoma
        res_decision = decidir_recoleccion(res_boot.L, res_boot.U,
            ...)

        # 7. Reporte integrado
        return ReporteAgente(V_hat, res_boot, res_mixing, ...)
```

7.2. Comportamiento Adaptativo

El orquestador exhibe comportamiento adaptativo en dos aspectos clave:

1. **Activación condicional del Delta Method**: Solo se ejecuta si el usuario proporciona Q^π, π y $\mathbf{\Sigma}_Q$. En caso contrario, se omite.

2. **Bootstrap por bloques automático**: Si el diagnóstico de mixing falla ($\hat{\rho} \geq 0{,}95$), el agente activa automáticamente el bootstrap por bloques con tamaño heurístico $\ell \approx n^{1/3}$.

7.3. Reporte Integrado

El reporte final (`ReporteAgente`) incluye:

- **Estimación puntual**: $\hat{V}^{\pi}$ con su error estándar.
- **IC Bootstrap**: Límites inferior y superior, ancho, y tipo de IC utilizado.
- **Diagnóstico de mixing**: $\hat{\rho}$, veredicto, y varianza de largo plazo.
- **Descomposición**: Coeficientes epistémico y aleatorio, fracción actual.
- **Delta Method** (si aplica): Varianza asintótica, contribución de covarianzas.
- **Decisión**: `actuar` o `recolectar_más`, con justificación.
- **Garantías teóricas**: Lista de teoremas y proposiciones que respaldan cada resultado.

7.3.1. Ejemplo de Salida

Listing 7: Ejemplo de reporte generado por el agente

```
================================================================================

REPORTE DEL AGENTE ESTADISTICO ASINTOTICO
================================================================================

Estimacion puntual V^pi      : 4.8200
IC Bootstrap (percentil-t):
  [4.0812, 5.5623] (nivel 95%)
  Ancho IC                   : 1.4811
Diagnostico Mixing           : rho = 0.1234 (OK)
Varianza de largo plazo      : 14.2891
Decision                     : ACTUAR
  Justificacion              : Ancho IC (1.4811)  tolerancia
    (0.5000)
--------------------------------------------------------------------------------

Garantias teoricas activas:
  * prop:ch5-clt-retornos -- CLT para retornos Monte Carlo
  * thm:ch5-clt-agentes -- CLT con varianza de largo plazo
  * thm:ch5-bootstrap-media -- consistencia bootstrap
  * def:ch5-ic-t -- IC percentil-t
  * thm:ch5-cobertura -- error de cobertura O(n^{-1})
  * Lab F.5.2 -- descomposicion epistemica vs aleatoria
  * Lab F.5.1 -- criterio de ancho para decision
================================================================================
```

8. Supuestos Críticos y Limitaciones

8.1. Supuestos No Verificables

El README del agente identifica explícitamente cinco supuestos críticos que el agente no puede verificar automáticamente:

Cuadro 4: Supuestos críticos no verificables.

Supuesto	Afecta a	Riesgo si se viola
$\mathbb{E}[G^2] < \infty$	Bootstrap, CLT	Inconsistencia del bootstrap
Estacionariedad estricta	Diagnóstico mixing	Falsos positivos/negativos
iid subyacente	IC Percentil	IC mal calibrados
Diferenciabilidad de g	Delta Method	Varianza asintótica incorrecta
Homocedasticidad	Descomposición	Separación errónea

8.2. Limitaciones Prácticas

1. **Costo computacional del percentil-t**: El doble bootstrap requiere $B \times B_{\text{inner}}$ operaciones, haciéndolo computacionalmente intensivo para $B \geq 1000$.

2. **Muestras pequeñas** ($n < 30$): El CLT puede no aproximar bien la distribución, y el bootstrap puede ser errático.

3. **Dependencia de largo alcance**: El bootstrap por bloques requiere que $\ell_n \to \infty$ y $\ell_n/n \to 0$, lo que puede ser difícil de satisfacer en la práctica.

4. **Sensibilidad al parámetro** ε: La decisión de actuar o recolectar más datos depende crucialmente de la tolerancia proporcionada por el usuario.

8.3. Recomendaciones de Uso

- Usar $B \geq 1000$ réplicas bootstrap (recomendación del Thm 5.7).
- Asegurar $n \geq 30$ trayectorias para validez del CLT.
- Para entornos con dependencia temporal, verificar el diagnóstico de mixing antes de interpretar los IC.
- La descomposición epistémica/aleatoria es más fiable con $n \geq 500$.

9. Mapeo Completo entre Teoría e Implementación

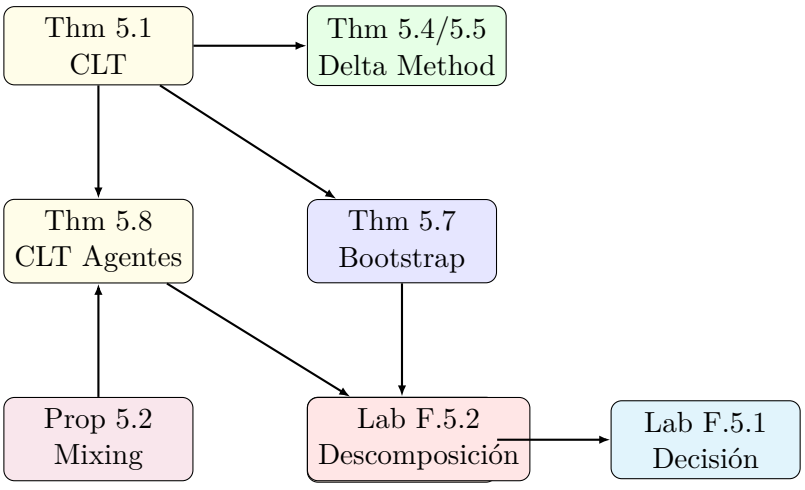
9.1. Correspondencia Detallada

La tabla 5 muestra la correspondencia entre los conceptos teóricos del Capítulo 5 y su implementación en el código.

Cuadro 5: Mapeo completo teoría → implementación.

Concepto Teórico	Referencia	Función/Clase	Líneas
CLT clásico	Thm 5.1	Fundamento de bootstrap_ci	120–130
Método Delta univariado	Thm 5.4	propagar_desde_Q (transf.)	130–150
Método Delta multivariado	Thm 5.5	propagar_desde_Q	100–130
Consistencia bootstrap	Thm 5.7	bootstrap_ci	130–200
CLT para agentes	Thm 5.8	diagnosticar_mixing	80–120
CLT retornos MC	Prop 5.2	AgenteEstadisticoAsintotico	20–30
Delta Q → V	Prop 5.4	propagar_desde_Q	130–150
Verificación mixing	Prop 5.2	diagnosticar_mixing	50–80
IC Percentil	Def 5.3	bootstrap_ci (tipo=”percentil”)	145–155
IC Percentil-t	Def 5.4	bootstrap_ci (tipo=”percentil-t”)	155–185
Cobertura bootstrap	Thm 5.7	bootstrap_ci (documentación)	-
Lab Decisión	F.5.1	decidir_recoleccion	50–80
Lab Descomposición	F.5.2	descomponer_incertidumbre	80–120

9.2. Diagrama de Dependencias Teóricas



10. Casos de Uso y Ejecución

10.1. Caso SSCH (Sistema de Soporte Clínico de Hígado)

El archivo `ejemplo_ssch.py` demuestra el uso del agente con el entorno SSCH:

- **Configuración**: $n = 100$, $\alpha = 0{,}05$, $\varepsilon = 0{,}5$, tipo IC = percentil-t, $B = 1000$.
- **Política**: π_b (práctica habitual).
- **Datos**: Retornos simulados como $N(4{,}82, 3{,}78^2)$.
- **Salida esperada**: IC bootstrap 95 % $\approx [4{,}08, 5{,}56]$ (ancho $\approx 1{,}48$).

10.2. Validación Cruzada con Valores del Capítulo

El script de ejemplo valida los resultados contra los valores esperados:

```
# Valores del capítulo
L_esperado, U_esperado = 4.08, 5.56
ancho_esperado = U_esperado - L_esperado  # 1.48

# Validación
rb = reporte.resultado_bootstrap
diff_L = abs(rb.L - L_esperado)
diff_U = abs(rb.U - U_esperado)

if diff_L < 0.5 and diff_U < 0.5:
    print("IC dentro de tolerancia")
else:
    print("IC difiere (variabilidad muestral esperada)")
```

10.3. Caso con Dependencia Temporal

El segundo demo del bloque principal utiliza una cadena de Markov de 2 estados con $\rho = 0{,}5$, demostrando el diagnóstico de mixing y la activación automática del bootstrap por bloques cuando es necesario.

10.4. Caso con Delta Method

El tercer demo propaga incertidumbre desde Q hasta V utilizando el MDP de 3 estados y 2 acciones, validando que la varianza asintótica coincide con el valor teórico $\sigma_V^2 = 3{,}456$.

11. Conclusiones

11.1. Resumen de Capacidades

El **Agente Estadístico Asintótico** implementa de manera operacional todos los componentes teóricos del Capítulo 5, proporcionando un pipeline completo para:

1. **Estimar** el valor de una política con rigor estadístico.

2. **Cuantificar** la incertidumbre mediante intervalos de confianza bootstrap de segundo orden.

3. **Diagnosticar** la presencia de dependencia temporal que pueda invalidar los supuestos del CLT.

4. **Propagar** la incertidumbre de manera correcta a través de transformaciones.

5. **Separar** las fuentes de incertidumbre en reducible (epistémica) e irreducible (aleatoria).

6. **Decidir** autónomamente si la precisión es suficiente para actuar.

11.2. Fortalezas del Diseño

- **Trazabilidad teórica**: Cada función documenta explícitamente qué teoremas y definiciones implementa.
- **Modularidad**: Los seis módulos son funcionalmente independientes y reutilizables.
- **Contratos explícitos**: Las dataclasses proporcionan interfaces claras de entrada/-salida.
- **Comportamiento adaptativo**: El agente ajusta su metodología (e.g., bootstrap por bloques) según las características de los datos.
- **Honestidad epistémica**: Documenta explícitamente los supuestos que no puede verificar.

11.3. Líneas de Extensión Futura

1. **Bootstrap suave** (smooth bootstrap): Para mejorar el rendimiento con muestras pequeñas.

2. **Subsampling**: Alternativa computacionalmente más eficiente que el bootstrap por bloques para datos dependientes.

3. **IC bootstrap BCa** (bias-corrected and accelerated): Para mejorar la cobertura en presencia de sesgo.

4. **Métodos de remuestreo secuencial**: Para entornos donde el costo de recolección varía.

5. **Diagnóstico automático de estacionariedad**: Para complementar el diagnóstico de mixing.